MODEL DEVELOPMENT DOCUMENT

Predictive Maintenance - Machine Failure Classification

Submitted by

VINEETH M

Applied DS, ML & AI

E & ICT Academy, IIT Guwahati

Capstone Project 1

July 2026

1. Project Overview

1.1 Project Details

Project Title: Predictive Maintenance - Machine Failure Classification

Project Type: Capstone Project 1 - E & ICT Academy, IIT Guwahati

Submitted By: Vineeth M

Domain: Industrial IoT / Healthcare Equipment Maintenance

Dataset: Predictive Maintenance Classification Dataset (10,000 records)

Tools Used: Python, VS Code, Jupyter Notebook, scikit-learn, XGBoost, imbalanced-learn, Streamlit

Deployment: Streamlit Cloud: https://predictive-maintenance-by-vineeth.streamlit.app/

GitHubt: https://github.com/Vineeth-Muraleedharan

1.2 Problem Statement

Unplanned equipment failure is one of the costliest operational challenges in industrial and clinical settings. Traditional maintenance strategies, such as reactive (fixing after failure) and preventive (scheduled maintenance regardless of condition), are either too late or too wasteful. Predictive maintenance represents a data-driven alternative: using real-time sensor data to anticipate failure before it occurs.

This project develops a machine learning classification model to predict whether a machine will fail based on five sensor readings: air temperature, process temperature, rotational speed, torque, and tool wear time. The binary target variable (0 = No Failure, 1 = Failure) makes this a supervised classification problem.

The primary technical challenge is severe class imbalance: only 3.39% of records represent failure events, with the remaining 96.61% being normal operations. This imbalance, if not addressed, causes models to default predicting "no failure" for every input, achieving high apparent accuracy but failing to detect any actual failures.

1.3 Clinical and Industrial Relevance

While the dataset originates from general industrial machinery, the methodology is directly applicable to medical equipment maintenance also. For example, In radiation therapy centers, critical equipment including:

- Linear accelerators (linacs)
- CT simulators
- MRI scanners and treatment planning workstations

...must maintain operational continuity. Equipment downtime directly impacts patient treatment schedules, delays fractions in curative radiotherapy, and in some cases affects treatment outcomes. A predictive maintenance system that flags impending failure 24-48 hours in advance allows scheduled maintenance without disrupting clinical workflows.

1.4 Project Objectives

- Perform comprehensive EDA to understand feature distributions and failure patterns
- Apply outlier detection and treatment using the IQR method
- Engineer three new features from existing sensor data to improve model discrimination
- Address class imbalance using SMOTE (Synthetic Minority Oversampling Technique)
- Train and compare six ML classification models
- Tune the best model using GridSearchCV
- Deploy the final model as an interactive Streamlit web application

2. Dataset Description

2.1 Dataset Overview

The dataset contains 10,000 records of machine operational sensor readings with a binary failure indicator. Each row represents one operational cycle of a machine.

Attribute	Details
Source	IIT Guwahati Capstone Project Material
Total Records	10,000 rows
Total Features	9 columns (7 input + 1 ID + 1 target)
Target Variable	Target (0 = No Failure, 1 = Failure)
Missing Values	None
Duplicate Rows	None
Failure Rate	3.39% (339 failures out of 10,000)

2.2 Feature Description

Feature	Type	Range	Description
UDI	Integer	1–10000	Unique identifier - dropped during modelling
Product ID	String	L/M/H + number	Product identifier - dropped during modelling
Type	Categorical	L, M, H	Machine type: Light, Medium, Heavy-duty
Air temperature [K]	Continuous	295.3 - 304.5	Ambient air temperature in Kelvin
Process temperature [K]	Continuous	305.7 - 313.8	Process operating temperature in Kelvin
Rotational speed [rpm]	Continuous	1168 - 2886	Motor rotational speed in RPM
Torque [Nm]	Continuous	3.8 - 76.6	Applied torque in Newton-meters
Tool wear [min]	Integer	0 - 253	Cumulative tool usage time in minutes
Target	Binary	0, 1	Target: 0 = No Failure, 1 = Failure

3. Exploratory Data Analysis (EDA)

3.1 Class Distribution Analysis

The first step in EDA was examining the target variable distribution. The dataset revealed a severe class imbalance:

- No Failure (Class 0): 9,661 records - 96.61%
- Failure (Class 1): 339 records - 3.39%
- Class ratio: approximately 28.5: 1

This extreme imbalance is the central challenge of this project. A naive classifier that always predicts "no failure" would achieve 96.61% accuracy, appearing excellent while being completely useless for detecting failures. This finding immediately justified the need for SMOTE and careful selection of evaluation metrics beyond accuracy.

3.2 Univariate Analysis - Feature Distributions

Each numerical feature was examined through histograms and boxplots, stratified by failure status (failure vs. no failure):

Air Temperature [K]

Approximately normally distributed with mean ~300K and range 295 - 305K. The distributions for failure and non-failure cases show slight overlap with failures occurring at slightly higher temperatures. Low standalone predictive power (correlation with target: 0.083).

Process Temperature [K]

Similarly, normally distributed with mean ~310K and range 306 - 314K. Very low correlation with target (0.036), suggesting process temperature alone is insufficient to predict failure. However, its difference from air temperature (Temp_Diff) proved more informative.

Rotational Speed [rpm]

Right-skewed distribution with mean ~1539 rpm and notable outliers above 2500 rpm. Negative correlation with failure (-0.044), machines running at extremely high speeds appear to fail less frequently, possibly because high-speed operation is only possible when the machine is healthy. This counterintuitive finding was preserved rather than corrected.

Torque [Nm]

The strongest individual predictor of failure (correlation: 0.191). Failure cases show notably higher torque values (mean ~60Nm) compared to non-failure cases (mean ~38Nm). The physical explanation: excessive torque indicates overloading of the machine mechanism, a common precursor to mechanical failure.

Tool Wear [min]

Uniformly distributed from 0 to 253 minutes with correlation of 0.105 with failure. Longer tool usage increases failure probability, which is physically intuitive. The interaction between tool wear and torque (engineered as Wear_Torque) proved more predictive than tool wear alone.

3.3 Categorical Analysis - Machine Type

Failure rate was analyzed by machine type (L/M/H):

Machine Type	Count	Failures	Failure Rate
L - Light Duty	6,000	~196	~3.27%
M - Medium Duty	2,997	~104	~3.47%
H - Heavy Duty	1,003	~39	~3.89%

Heavy duty machines show slightly higher failure rates as expected. Machine type was Label Encoded (L=0, M=1, H=2) to preserve the ordinal relationship between machine duty levels.

3.4 Bivariate Analysis - Failure Zones

Scatter plots of key feature pairs revealed distinct failure zones:

- Tool Wear vs Torque: Failures cluster at high torque (>60Nm) AND high wear (>200min), confirming that the combination of overloading and wear accelerates failure
- Rotational Speed vs Torque: Failures cluster at extreme speed-torque combinations, consistent with mechanical stress theory
- These visual patterns directly motivated the creation of engineered features (Wear_Torque, Power)

3.5 Correlation Analysis

A correlation matrix was computed for all numerical features against the target:

Feature	Correlation with Target	Interpretation
Torque [Nm]	0.191	Strongest predictor - high torque = overloading
Wear_Torque *	0.173	Engineered: load-weighted wear
Tool wear [min]	0.105	Longer wear increases risk
Power *	0.089	Engineered: mechanical power proxy
Air temperature [K]	0.083	Weak - temperature alone insufficient
Temp_Diff *	0.061	Engineered: thermal stress indicator
Process temperature [K]	0.036	Very weak standalone predictor
Rotational speed [rpm]	-0.044	Negative - high speed = healthy machine

* Engineered features. Their competitive correlations validate the feature engineering approach.

4. Data Preprocessing

4.1 Handling Missing Values and Duplicates

Data quality check confirmed no missing values and no duplicate records across all 10,000 rows. This is characteristic of sensor-generated industrial data where readings are systematically recorded. No imputation was required.

4.2 Skewness Analysis

Skewness was computed for all numerical features to determine the need for transformations:

Feature	Skewness	Classification	Action
Air temperature	+0.023	Approximately normal	None needed
Process temperature	-0.017	Approximately normal	None needed
Rotational speed	+0.897	Moderate positive skew	Winsorize outliers
Torque	+0.121	Approximately normal	Winsorize outliers
Tool wear	+0.009	Approximately normal	None needed

Log transformation was not applied because the skewness levels were moderate and StandardScaler (applied in Section 4.5) handles this adequately by normalising the distributions. Applying log transformation before StandardScaler can distort feature relationships, particularly for features where zero values are meaningful.

4.3 Outlier Detection and Treatment

Detection Method: IQR (Interquartile Range)

The IQR method was chosen over Z-score for outlier detection because:

- Z-score assumes normal distribution, not appropriate for all features
- IQR is robust to the presence of extreme outliers (Z-score is influenced by the very outliers it tries to detect)
- IQR works well for skewed distributions like Rotational Speed

Feature	Outliers Found	% of Data	Action
Air temperature	0	0.00%	None required
Process temperature	0	0.00%	None required
Rotational speed	418	4.18%	Winsorization applied
Torque	69	0.69%	Winsorization applied
Tool wear	0	0.00%	None required

Treatment Method: Winsorization

Winsorization clips extreme values to the 1st and 99th percentile boundaries rather than removing them. This approach was chosen over:

- Removal: Removing 418 rows (4.18%) risks losing failure cases that are precisely characterized by extreme sensor values
- Mean/median imputation: Replaces meaningful extreme values with central tendency, destroying failure zone information
- Winsorization preserves all 10,000 records while limiting the distorting influence of extreme values on model training

4.4 Feature Engineering

Three new features were derived from domain knowledge of mechanical systems:

Temp_Diff = Process Temperature - Air Temperature

The temperature differential between the process and ambient environment is a proxy for thermal stress on the machine. A large differential suggests the machine is generating excessive heat, a well-established precursor to mechanical failure. This feature captures the interaction between two otherwise weakly predictive individual temperatures.

Power = Torque × Rotational Speed

Mechanical power (in arbitrary units) is the product of torque and rotational speed. This is derived from the fundamental physics of rotary machines: $P = \tau\omega$. High power consumption indicates the machine is under load stress, combining two features that individually have different relationships with failure.

Wear_Torque = Tool Wear × Torque

This interaction term captures the compounding effect of tool wear under load. A machine operating at high torque with heavily worn tools is significantly more failure-prone than either condition alone. The product creates a non-linear feature that reflects this real-world interaction.

Justification for feature engineering: The correlation analysis showed these engineered features (0.173, 0.089, 0.061) competitive with original features, confirming their added predictive value.

4.5 Categorical Encoding

Machine Type (L/M/H) was encoded using Label Encoding rather than One-Hot Encoding. Justification:

- The three machine types represent ordered duty levels: Light < Medium < Heavy
- Label encoding (L=0, M=1, H=2) preserves this ordinal relationship
- One-hot encoding would create three binary columns and lose the ordinal information
- Tree-based models (Random Forest, XGBoost, Gradient Boosting) handle label-encoded ordinal features correctly

4.6 Train-Test Split

The dataset was split 80:20 (8,000 training / 2,000 test records) using stratified splitting:

- Stratified on target variable - ensures both splits maintain the 3.39% failure rate
- Train failure rate: 3.39% | Test failure rate: 3.40%, nearly identical
- random_state=42 for reproducibility

80:20 was chosen over 70:30 because with only 339 total failures, maximising training data (272 training failures vs 238 in 70:30) improves SMOTE quality and model learning.

4.7 Feature Scaling - StandardScaler

Why StandardScaler over MinMaxScaler?

StandardScaler (z-score normalisation) was chosen over MinMaxScaler for the following reasons:

Criteria	StandardScaler	MinMaxScaler
Formula	(x − μ) / σ	(x − min) / (max − min)
Output range	~ [-3, 3] (unbounded)	[0, 1] (bounded)
Outlier sensitivity	Robust - outliers shift mean/std mildly	Sensitive - outliers compress all other values
Distribution assumption	Works for any distribution	Assumes uniform distribution
Best for	Tree models, SVM, Logistic Regression	Neural networks, KNN (specific cases)

The dataset contains outliers in Rotational Speed (4.18%) and Torque (0.69%). MinMaxScaler compresses all values into [0,1] based on the observed minimum and maximum. Any outlier becomes the 0 or 1 boundary, squashing the majority of data into a narrow range and making it harder for models to discriminate. StandardScaler is more robust because mean and standard deviation are less affected by individual extreme values.

Additionally, StandardScaler is the standard choice when using SVM (which is distance-based and assumes zero-mean, unit-variance features) and Logistic Regression (gradient descent converges faster with normalised features). Since both models were included in the comparison, StandardScaler was the appropriate choice for a fair comparison.

5. Handling Class Imbalance - SMOTE

5.1 Why Class Imbalance is Critical

With 96.61% of records being non-failures, a model trained without addressing this imbalance will learn to predict "no failure" for every input simply. While this achieves 96.61% accuracy, it has 0% recall for the failure class - completely useless for the intended application. In predictive maintenance, missing a failure (false negative) is catastrophically more costly than a false alarm (false positive).

5.2 SMOTE - Synthetic Minority Oversampling Technique

SMOTE was selected over other approaches:

Approach	Method	Reason Not Chosen
Random oversampling	Duplicate minority samples	Exact copies cause overfitting
Random undersampling	Remove majority samples	Wastes 90%+ of data; loses information
Class weights	Penalize the majority class errors more	Less effective with extreme imbalance (28:1)
SMOTE (chosen)	Synthesize new minority samples	Creates diverse new data; avoids overfitting

SMOTE generates synthetic minority class samples by interpolating between existing minority class data points in feature space. For each minority sample, it selects k nearest neighbors and creates new points along the line segments connecting them.

5.3 SMOTE Results

Class	Before SMOTE	After SMOTE	Change
No Failure (0)	7,729	7,729	No change
Failure (1)	271	7,729	+7,458 synthetic samples
Total Training	8,000	15,458	+7,458 records
Ratio	28.5:1	1:1	Perfectly balanced

Critical note: SMOTE was applied only to the training data AFTER the train-test split. The test set retains the original 3.40% failure rate, ensuring evaluation reflects real-world performance conditions.

6. Model Development

6.1 Model Selection Rationale

Six classification algorithms were selected to cover a broad spectrum of approaches:

Logistic Regression (Baseline)

A linear probabilistic classifier included as the baseline. It assumes linear separability of classes in feature space. While unlikely to perform optimally on this non-linear problem, it provides a lower-bound performance reference. Fast to train and highly interpretable.

Random Forest

An ensemble of decision trees using bagging (Bootstrap Aggregating). Each tree is trained on a random subset of data and features. The ensemble aggregation reduces variance and overfitting. Particularly suitable for tabular data with mixed feature types and handles non-linear relationships well.

XGBoost (Extreme Gradient Boosting)

A gradient boosting implementation optimized for speed and performance. Trees are built sequentially, each correcting error of the previous. Includes L1/L2 regularization to prevent overfitting. Consistently performs well on structured/tabular data and was expected to be among the top performers.

Support Vector Machine (SVM)

A kernel-based algorithm that finds the optimal hyperplane maximising the margin between classes. The RBF (Radial Basis Function) kernel was used to handle non-linear boundaries. SVM is particularly effective in high-dimensional spaces but is computationally intensive. Probability = True enables probability calibration for ROC - AUC computation.

K-Nearest Neighbours (KNN)

An instance-based algorithm that classifies based on the majority class among the k nearest training examples (k=5). No training phase: prediction is computationally expensive. Included to provide a non-parametric baseline. Sensitive to feature scaling, hence StandardScaler is critical for this model.

Gradient Boosting

A sequential ensemble method that builds trees to minimize a differentiable loss function. Unlike XGBoost, this is the native scikit-learn implementation. Typically produces fewer extreme predictions and may generalize better on imbalanced data despite SMOTE balancing. Selected as the final best model based on recall performance.

6.2 Evaluation Metrics Selection

Given the class imbalance context, multiple metrics were used:

Metric	Formula	Why Used
Recall (Sensitivity)	TP / (TP + FN)	Primary: proportion of failures caught - missing a failure is most costly
Precision	TP / (TP + FP)	Secondary: false alarm rate - unnecessary maintenance has cost
F1 Score	2 × (P × R) / (P + R)	Harmonic mean of Precision and Recall - balances both
ROC-AUC	Area under ROC curve	Threshold-independent measure of discrimination
Accuracy	Correct / Total	Reported but NOT primary - misleading with imbalance

6.3 Training Configuration

All models were trained on the SMOTE-balanced training data (15,458 samples) and evaluated on the original test data (2,000 samples with 3.40% failures). This ensures evaluation reflects deployment conditions. random_state=42 was set for all stochastic models to ensure reproducibility.

7. Model Results and Comparison

7.1 Performance Summary

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gradient Boosting	94.60%	0.3750	0.8824	0.5263	0.9746
XGBoost	97.20%	0.5652	0.7647	0.6500	0.9712
Random Forest	96.90%	0.5300	0.7794	0.6310	0.9704
SVM	92.10%	0.2857	0.8824	0.4317	0.9659
Logistic Regression	85.75%	0.1742	0.8529	0.2893	0.9311
KNN	93.05%	0.2924	0.7353	0.4184	0.8729

(Highlighted row = selected best model)

7.2 Key Findings

Finding 1: Gradient Boosting achieves highest Recall and ROC - AUC

Gradient Boosting detected 88.24% of all actual failures, meaning out of 68 test failures, 60 were correctly identified. This is the most operationally important outcome: only 8 failures were missed. Its ROC - AUC of 0.9746 indicates excellent discrimination across all classification thresholds.

Finding 2: Accuracy is misleading

Logistic Regression (85.75% accuracy) might appear poor, but its recall (0.8529) is competitive. Conversely, XGBoost (97.20% accuracy) looks superior, but its recall (0.7647) means it misses 23.5% of failures. Accuracy optimizes for the majority class; recall optimizes for the minority class where the real value lies.

Finding 3: SVM ties Gradient Boosting on Recall but lower AUC

SVM achieved identical recall (0.8824) to Gradient Boosting but with much lower precision (0.2857 vs 0.3750) and lower ROC-AUC (0.9659 vs 0.9746). This means SVM generates more false alarms per true detection. Gradient Boosting provides a better balance.

Finding 4: KNN is the weakest performer

KNN achieved the lowest ROC-AUC (0.8729), indicating poor discrimination capability. Instance-based learning struggles with the sparse failure class even after SMOTE, as the local neighbourhood structure in 9-dimensional space is difficult to exploit effectively.

7.3 Hyperparameter Tuning - XGBoost

GridSearchCV was applied to XGBoost with 3-fold cross-validation across the following parameter grid:

- n_estimators: [100, 200]
- max_depth: [3, 5, 7]
- learning_rate: [0.01, 0.1, 0.2]
- subsample: [0.8, 1.0]

The tuned XGBoost achieved 97.30% accuracy, 0.7206 recall, and 0.9711 ROC - AUC. While accuracy improved marginally, recall decreased from 0.7647 to 0.7206 compared to the default XGBoost, confirming that Gradient Boosting remains the superior choice for this use case.

8. Final Model Selection and Justification

8.1 Selected Model: Gradient Boosting Classifier

Gradient Boosting was selected as the final deployment model based on the following justification:

Criterion	Value	Justification
Recall	0.8824	Highest - catches 88.24% of all failures
ROC-AUC	0.9746	Highest - best overall discrimination
Precision	0.3750	Acceptable - 37.5% of alarms are real failures
F1 Score	0.5263	Balanced harmonic mean
False Negatives	8 out of 68	Only 8 failures missed in test set

8.2 Operational Interpretation

On the 2,000-record test set (68 actual failures):

- True Positives (failures correctly caught): 60
- False Negatives (failures missed): 8
- False Positives (false alarms): ~100
- True Negatives (correct normal predictions): ~1,832

In operational terms: for every 160 maintenance alerts generated by the model, approximately 60 are genuine failures detected early and 100 are unnecessary checks. Given that one undetected failure costs

orders of magnitude more than an unnecessary maintenance check, this trade-off is clinically and industrially acceptable.

8.3 Prediction Threshold Adjustment

The default classification threshold of 0.5 was lowered to 0.3 in the deployed Streamlit application. This is consistent with predictive maintenance best practices:

- At threshold 0.5: standard binary classification
- At threshold 0.3: earlier warnings for machines approaching failure zone
- Three-tier output: High Risk (≥50%), Early Warning (30-50%), Normal (<30%)

This adjustment prioritizes recall (catching failures early) over precision (reducing false alarms), which is appropriate when the cost of missing a failure exceeds the cost of unnecessary inspection.

8.4 Feature Importance Analysis

Feature importance from the Random Forest model (most interpretable ensemble):

Rank	Feature	Importance	Type
1	Wear_Torque	Highest	Engineered
2	Torque [Nm]	High	Original
3	Power	High	Engineered
4	Tool wear [min]	Medium	Original
5	Rotational speed [rpm]	Medium	Original
6	Temp_Diff	Low-Medium	Engineered
7	Air temperature [K]	Low	Original
8	Process temperature [K]	Low	Original
9	Type	Low	Original (encoded)

The dominance of engineered features (Wear_Torque, Power) at the top validates the feature engineering decisions. Torque-related features (Torque, Wear_Torque, Power) account for most of the predictive power, consistent with domain knowledge that mechanical overloading is the primary failure mechanism.

9. Model Deployment

9.1 Streamlit Web Application

The final Gradient Boosting model was deployed as an interactive web application using Streamlit. The application allows real-time failure risk prediction by inputting current sensor readings.

9.2 Application Architecture

- Model loading: Pre-trained model loaded from pickle file (best_maintenance_model.pkl) at startup using @st.cache_resource for efficiency
- Feature computation: Engineered features (Temp_Diff, Power, Wear_Torque) are automatically computed from raw sensor inputs

- Prediction: Inputs are scaled using the saved StandardScaler, then passed to the Gradient Boosting model
- Output: Three-tier risk classification with probability gauge chart

9.3 Application Features

- Interactive sensor input sliders and number inputs
- Real-time failure probability gauge (0-100%)
- Three-tier risk classification: High Risk / Early Warning / Normal
- Expandable section showing auto-computed engineered features
- Hosted on Streamlit Cloud at: (https://predictive-maintenance-by-vineeth.streamlit.app/)

9.4 Deployment Files

File	Purpose
app.py	Streamlit application code
best_maintenance_model.pkl	Serialised Gradient Boosting model
scaler_maintenance.pkl	Serialised StandardScaler
code.ipynb	Complete analysis notebook
predictive_maintenance.csv	Original dataset
requirements.txt	Python dependencies

10. Conclusions and Future Work

10.1 Summary of Findings

This project successfully developed a predictive maintenance classification system capable of detecting machine failures with 88.24% recall and 0.9746 ROC-AUC. Key conclusions:

- SMOTE effectively addressed the 28.5:1 class imbalance, enabling all models to learn failure patterns
- Feature engineering added significant predictive value - engineered features ranked highest in importance
- Gradient Boosting outperformed XGBoost, Random Forest, SVM, KNN, and Logistic Regression on the primary metric (Recall)
- StandardScaler was the appropriate choice given the presence of outliers and the mixture of model types compared
- Torque and tool wear, both individually and in combination, are the strongest predictors of mechanical failure

10.2 Limitations

- Dataset is synthetic, real industrial sensor data may exhibit different distributions and noise characteristics
- Only 339 failure cases, a larger failure dataset would improve model robustness
- Failure type classification (TWF, HDF, PWF, OSF, RNF) was not explored, multi-class prediction is a natural extension

- No temporal/time-series features, sensor readings over time (trends, rate of change) could improve predictions

10.3 Future Work

- Implement LSTM/GRU time-series model to capture temporal sensor patterns
- Extend to multi-class failure type prediction
- Integrate with real-time IoT sensor streams (MQTT/Kafka pipeline)
- Apply methodology to radiation therapy equipment (linacs, CT simulators) with clinical validation
- Add SHAP (SHapley Additive exPlanations) values to explain individual predictions in the Streamlit app

11. References

- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
- Chawla, N. V., et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of AI Research, 16, 321-357.
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), 1189-1232.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825-2830.
- Scikit-learn Documentation: https://scikit-learn.org/stable/
- Streamlit Documentation: https://docs.streamlit.io/

VINEETH M

Email: vinuvineeth616@gmail.com

Mob: +91 7560946570

Milestones	Status & Description
Define a problem	Complete - Identified machine failure prediction as the problem using sensor data from industrial equipment
Understanding the business problem	Complete - Analyzed impact of unplanned downtime in industrial/clinical settings; defined success metrics (Recall, ROC-AUC)
Get the Data	Complete - Obtained predictive_maintenance.csv (10,000 records, 9 features) from IIT Guwahati capstone material
Explore and pre-process data	Complete - EDA performed; missing values checked; outliers detected (IQR) and treated (Winsorization); class imbalance identified (3.39% failures)
Choosing the Python platform	Complete - Python 3.10 selected with VS Code, Jupyter Notebook; libraries: scikit-learn, XGBoost, imbalanced-learn, Streamlit
Create Features	Complete - Three engineered features created: Temp_Diff, Power (Torque × RPM), Wear_Torque; Label Encoding for machine Type
EDA	Complete - Univariate and bivariate analysis; correlation matrix; failure zone scatter plots; SMOTE applied (28.5:1 → 1:1 balance)
Create Model	Complete - Six models trained: Logistic Regression, Random Forest, XGBoost, SVM, KNN, Gradient Boosting
Model Evaluation	Complete - Models compared on Recall, Precision, F1, ROC-AUC; Gradient Boosting selected (Recall: 0.8824, AUC: 0.9746); GridSearchCV tuning applied
Report Writing	Complete - Model Development Document (MDD) prepared with full justification for all decisions
Project submission	Complete - Jupyter notebook, Streamlit app, and MDD submitted; app deployed on Streamlit Cloud via GitHub